AirLens

Air Quality Intelligence Dashboard for Indian Cities

Submitted by:

Name : PRANAY KARTHIK

Reg. No : 25BCE10818

Table of Contents

Section 01

Problem Statement

Air quality in Indian cities has deteriorated significantly over the past decade. While data on pollutant concentrations is collected by the Central Pollution Control Board (CPCB) across hundreds of monitoring stations, this data is rarely made actionable for ordinary citizens, local administrators, or policymakers. Existing government portals display a current AQI value but offer no forecast, no explanation of which pollutants are responsible, and no comparison across cities.

This project asks:Can historical AQI data be used to forecast future air quality, identify the dominant pollution drivers, and reveal patterns that are invisible in day-by-day readings?

Section 02

Why This Problem Matters

India is home to 39 of the world's 50 most polluted cities according to recent WHO data. Exposure to PM2.5 and PM10 is linked to respiratory disease, cardiovascular conditions, and premature death. Bhopal, where this project was conceived, regularly records AQI values above 300 (classified as "Poor" to "Severe") during the winter months of November through January.

The problem is not a lack of data — CPCB publishes hourly readings. The problem is a lack ofintelligence on top of the data. A 30-day AQI forecast could help hospitals prepare for increased admissions, help schools decide on outdoor activity bans, or guide city-level traffic restrictions. A pollutant contribution breakdown could tell civic bodies whether to focus on industrial emissions (SO_2, NO_2) or vehicular traffic (PM2.5, CO).

This project is, in essence, a proof-of-concept for what a civic AI layer on top of public environmental data could look like.

Section 03

Dataset and Data Sources

Primary Dataset

The main dataset is theAir Quality Data in India (2015–2020)published on Kaggle, originally sourced from CPCB. It contains daily readings for 26 Indian cities across six pollutants: PM2.5, PM10, NO2, SO2, O3, and CO, along with a composite AQI value.

Weather Features (Supplementary)

Wind speed, temperature, and humidity are known to influence pollutant dispersion. The Open-Meteo API (free, no API key required) was used to fetch historical weather data for each city. In the current version, this is simulated; full integration is flagged as future scope.

Data Quality

The dataset has approximately 15% missing values, primarily in O3 and CO readings. Missing values were forward-filled within each city's time series. Outliers (AQI > 450) were retained as they represent genuine pollution events during festival seasons and crop-burning periods.

Section 04

Approach and Methodology

The project follows a standard ML pipeline with three parallel modelling tracks, all feeding into a unified dashboard.

Step 1 — Exploratory Data Analysis

Before modelling, I spent significant time in01_EDA.ipynbunderstanding the data. Key findings: PM2.5 shows the strongest correlation with AQI ($r \approx 0.82$ across cities), AQI peaks in December–January in North Indian cities (due to inversion layers trapping smoke from crop residue burning), and South Indian cities (Bangalore, Chennai) have structurally lower AQI year-round due to coastal wind patterns.

Step 2 — Feature Engineering

From the raw daily readings, I derived: month and day-of-week as cyclic sine/cosine features, a 7-day rolling average per pollutant, year-over-year delta, and AQI category as an ordinal label for the classifier.

Step 3 — Modelling (three tracks)

- Time-series forecasting (Prophet) — one model trained per city
- Category classification (Random Forest) — one global model across all cities
- Unsupervised clustering (K-Means) — city-level pollution profile segmentation

Step 4 — Dashboard Integration

All model outputs were wired into a Streamlit app. Streamlit's@st.cache_datadecorator ensures models are trained once per session, not on every interaction. A city dropdown in the sidebar drives the city-specific Prophet model and filters all other charts to the selected city.

Section 05

Machine Learning Models

5.1 — Prophet (AQI Forecasting)

Facebook Prophet is a decomposable time-series model that explicitly models trend, yearly seasonality, weekly seasonality, and holiday effects. It is well-suited to AQI data because AQI has strong annual seasonality (winter spikes, monsoon dips) and the historical series is noisy.

Configuration:yearly_seasonality=True,weekly_seasonality=True,changepoint_prior_scale=0.05(low to prevent overfitting to noise). The model outputs a 30-day forecast with 80% and 95% uncertainty intervals.

5.2 — Random Forest Classifier (Pollutant Contribution)

ARandomForestClassifierwith 100 estimators was trained on PM2.5, PM10, NO2, SO2, O3, and CO to predict AQI category (five classes). The model'sfeature_importances_attribute directly answers the question: which pollutants most influence the AQI classification?

SHAP (SHapley Additive exPlanations) values were computed to provide a more rigorous, instance-level explanation — not just global importance but which pollutant pushed a specific day's prediction in which direction.

5.3 — K-Means Clustering (City Profiles)

Monthly average AQI was computed for each city, creating a city × month matrix. After scaling withStandardScaler, K-Means with k=4 was applied. The optimal k was confirmed using the elbow method on inertia values (k=3 was close, k=4 gave a meaningfully lower within-cluster variance).

The four clusters broadly correspond to: high-pollution industrial North Indian cities, moderate-pollution tier-2 cities, low-pollution South Indian coastal cities, and cities with high seasonal variance (good summers, bad winters).

Section 06

Key Technical Decisions

Why Streamlit over Flask/Django?

Streamlit converts Python scripts into interactive web apps with minimal boilerplate. For a data-science capstone where the primary deliverable is the analysis, not the web framework, Streamlit let me move directly from model code to a deployable dashboard without writing HTML/CSS or REST endpoints. The tradeoff is limited layout control, which I partially addressed with Plotly's native styling.

Why one Prophet model per city?

AQI dynamics differ significantly between cities — Delhi's AQI is driven by crop burning in Punjab and Haryana, while Ahmedabad's is driven by industrial activity. A single global forecasting model would average out these local patterns. Training one model per city (10 cities in the current build) is feasible within Streamlit's caching layer.

Caching strategy

All three model training functions are decorated with@st.cache_data. Without caching, switching cities would trigger full Prophet retraining on every interaction (~15 seconds per city). With caching, subsequent selections are instant. The tradeoff is higher memory usage — acceptable for a local or single-user deployment.

Synthetic data fallback

Theload_data()function generates synthetic AQI data if no Kaggle CSV is present. This made the dashboard demonstrable during development without committing the 50MB dataset to the repository. The synthetic data matches the column schema exactly, so switching to real data requires only adding the CSV — no code changes.

Section 07

Dashboard Design

The dashboard is laid out as a single Streamlit page with a left sidebar for city selection. The main area is divided into three horizontal bands:

- Metric row— four KPI cards: current AQI, 30-day average, number of severe days, and 7-day forecast with delta indicator.
- Primary analysis row— the Prophet forecast chart (wide) alongside the pollutant contribution bar chart (narrow).
- Pattern row— the year-over-year AQI heatmap alongside the K-Means city cluster scatter chart.

All charts are rendered with Plotly, which provides zoom, pan, hover tooltips, and PNG export out of the box. The layout is fully responsive to window width viause_container_width=True.

A key UX decision was to make the city selector the single point of control — one dropdown updates every panel simultaneously. This matches the mental model of a user asking "what is happening inmycity?" rather than configuring multiple independent charts.

Section 08

Challenges Faced

Prophet installation on Windows

Prophet depends onpystan, which requires a C++ compiler. Installation on Windows failed twice before I found that the Conda package (conda install -c conda-forge prophet) includes pre-compiled binaries. I added this to the README to save future users the same trouble.

Missing data in the AQI dataset

Approximately 15% of readings were missing, with O3 and CO having the most gaps. Forward-filling worked well for short gaps (1–3 days) but introduced slight distortion over longer gaps. A more robust approach would be to interpolate using weather conditions as auxiliary information — flagged as future work.

K-Means sensitivity to k

The elbow method suggested k=3 or k=4, with no strongly dominant choice. I settled on k=4 after inspecting which cities fell into each cluster — four clusters produced a more geographically and climatically interpretable grouping than three. This highlighted an important lesson: unsupervised learning requires domain knowledge to evaluate.

SHAP values with multi-class output

shap.TreeExplainerreturns a list of SHAP matrices (one per class) for multi-class classifiers. I initially tried to use the matrix for a single class, which produced misleading feature rankings. The correct approach is to sum the absolute SHAP values across all classes to get an aggregate importance — I revised the code after identifying this bug during testing.

Section 09

Results and Observations

Forecasting accuracy

On a held-out 60-day test set (not used for training), the Prophet model achieved a Mean Absolute Error of approximately 18–25 AQI units across cities. Given that the AQI scale runs from 0 to 500, this is a reasonable result for a 30-day horizon. Forecast accuracy degrades beyond 14 days, as expected for time-series models without external covariate inputs.

Pollutant contribution

Across all cities, PM2.5 emerged as the dominant predictor of AQI category (importance score ~0.45), followed by PM10 (~0.28). NO2 ranked third in most cities but was the dominant driver in Chennai and Mumbai, reflecting vehicular density. SO2's importance was higher in industrial cities like Ahmedabad and Kanpur.

City clusters

The K-Means model identified four city clusters that broadly aligned with geographic and economic profiles: a high-pollution North Indian cluster (Delhi, Lucknow, Kanpur, Patna), a coastal low-pollution cluster (Mumbai, Chennai), a mid-tier cluster (Bhopal, Jaipur, Pune), and a volatile seasonal cluster with high winter spikes. This clustering is useful for cross-city policy benchmarking.

Section 10

What I Learned

Technical learnings

- Time-series models require careful thought about what constitutes a "train/test split" — random splits leak future information; you must split chronologically.
- Feature importance from Random Forest and SHAP values do not always agree. SHAP is more trustworthy for instance-level explanations; RF importance can be biased toward high-cardinality features.
- Streamlit's caching system is powerful but requires functions to be pure (same inputs → same outputs) and to not contain mutable state. I had to restructure two functions to comply.
- Prophet'schangepoint_prior_scaleis the most impactful hyperparameter. Too high and the trend overfits to every spike; too low and it misses real structural changes in the data.

Process learnings

- EDA is not a formality — the insight that Bhopal has extreme seasonal variance came from the heatmap and directly influenced the choice to show seasonality decomposition in the dashboard.
- Real datasets are messier than tutorials. Handling missing data, inconsistent city names, and timezone-naive timestamps consumed about 30% of total project time.

- Building for a non-technical audience (citizens, administrators) forces you to simplify model outputs into interpretable visuals, which is a distinct skill from building a model in a notebook.

Section 11

Future Scope

- Live data integration— Connect to the CPCB real-time API so the dashboard shows today's actual AQI rather than historical data.
- Weather as a covariate in Prophet— Add temperature, wind speed, and humidity as external regressors. Prophet supports this natively and it should improve forecast accuracy beyond 14 days.
- Alert system— Send an email or push notification when the 7-day forecast crosses a threshold (e.g., AQI > 300), using a scheduled cron job.
- Choropleth map— Replace the scatter cluster chart with an actual map of India where cities are colored by cluster or current AQI, using Plotly'spx.scatter_mapbox.
- Deep learning baseline— Compare Prophet's accuracy against an LSTM or Temporal Fusion Transformer trained on the same data.
- Mobile-responsive UI— Deploy to Streamlit Community Cloud and optimize the layout for mobile, since citizens are more likely to check air quality on a phone than a desktop.

Name: Pranay Karthik
Registration No.: 25BCE10818